

Chapter III

Module-specific rules

It is mandatory to use the standard containers to perform each exercise in this module.

Once a container is used you cannot use it for the rest of the module.



It is advisable to read the subject in its entirety before doing the exercises.



You must use at least one container for each exercise with the exception of exercise 02 which requires the use of two containers.

You must submit a **Makefile** for each program which will compile your source files to the required output with the flags **-Wall**, **-Wextra** and **-Werror**.

You must use **c++**, and your **Makefile** must not relink.

Your **Makefile** must at least contain the rules **\$(NAME)**, **all**, **clean**, **fclean** and **re**.

Chapter V

Exercise 00: Bitcoin Exchange

	Exercise: 00
Bitcoin Exchange	
Directory: <i>ex00/</i>	
Files to Submit: <code>Makefile</code> , <code>main.cpp</code> , <code>BitcoinExchange.{cpp, hpp}</code>	
Forbidden: None	

You have to create a program which outputs the value of a certain amount of bitcoin on a certain date.

This program must use a database in csv format which will represent bitcoin price over time. This database is provided with this subject.

The program will take as input a second database, storing the different prices/dates to evaluate.

Your program must respect these rules:

- The program name is `btc`.
- Your program must take a file as an argument.
- Each line in this file must use the following format: `"date | value"`.
- A valid date will always be in the following format: Year-Month-Day.
- A valid value must be either a float or a positive integer, between 0 and 1000.



You must use at least one container in your code to validate this exercise. You should handle possible errors with an appropriate error message.

Here is an example of an input.txt file:

```
$> head input.txt
date | value
2011-01-03 | 3
2011-01-03 | 2
2011-01-03 | 1
2011-01-03 | 1.2
2011-01-09 | 1
2012-01-11 | -1
2001-42-42
2012-01-11 | 1
2012-01-11 | 2147483648
$>
```

Your program will use the value in your input file.

Your program should display on the standard output the result of the value multiplied by the exchange rate according to the date indicated in your database.



If the date used in the input does not exist in your DB then you must use the closest date contained in your DB. Be careful to use the lower date and not the upper one.

The following is an example of the program's use.

```
$> ./btc
Error: could not open file.
$> ./btc input.txt
2011-01-03 => 3 = 0.9
2011-01-03 => 2 = 0.6
2011-01-03 => 1 = 0.3
2011-01-03 => 1.2 = 0.36
2011-01-09 => 1 = 0.32
Error: not a positive number.
Error: bad input => 2001-42-42
2012-01-11 => 1 = 7.1
Error: too large a number.
$>
```



Warning: The container(s) you use to validate this exercise will no longer be usable for the rest of this module.

Chapter VI

Exercise 01: Reverse Polish Notation

	Exercise: 01
RPN	
Directory: <i>ex01/</i>	
Files to Submit: <code>Makefile</code> , <code>main.cpp</code> , <code>RPN.{cpp, hpp}</code>	
Forbidden: None	

You must create a program with these constraints:

- The program name is RPN.
- Your program must take an inverted Polish mathematical expression as an argument.
- The numbers used in this operation and passed as arguments will always be less than 10. The calculation itself but also the result do not take into account this rule.
- Your program must process this expression and output the correct result on the standard output.
- If an error occurs during the execution of the program an error message should be displayed on the standard error.
- Your program must be able to handle operations with these tokens: "+ - / *".



You must use at least one container in your code to validate this exercise.



You don't need to handle brackets or decimal numbers.

Here is an example of standard usage:

```
$> ./RPN "8 9 * 9 - 9 - 9 - 4 - 1 +"  
42  
$> ./RPN "7 7 * 7 -"  
42  
$> ./RPN "1 2 * 2 / 2 * 2 4 - +"  
0  
$> ./RPN "(1 + 1)"  
Error  
$>
```



Warning: The container(s) you used in the previous exercise are forbidden here. The container(s) you used to validate this exercise will not be usable for the rest of this module.

Chapter VII

Exercise 02: PmergeMe

	Exercise: 02
PmergeMe	
Directory: <i>ex02/</i>	
Files to Submit: Makefile, main.cpp, PmergeMe.{cpp, hpp}	
Forbidden: None	

You must create a program with these constraints:

- The name of the program is PmergeMe.
- Your program must be able to use a positive integer sequence as an argument.
- Your program must use the merge-insert sort algorithm to sort the positive integer sequence.



To clarify, yes, you need to use the Ford-Johnson algorithm.
(source: [Art Of Computer Programming, Vol.3](#). Merge Insertion,
Page 184.)

- If an error occurs during program execution, an error message should be displayed on the standard error.



You must use at least two different containers in your code to validate this exercise. Your program must be able to handle at least 3000 different integers.



It is strongly advised to implement your algorithm for each container and thus to avoid using a generic function.

Here are some additional guidelines on the information you should display line by line on the standard output:

- On the first line you must display an explicit text followed by the unsorted positive integer sequence.
- On the second line you must display an explicit text followed by the sorted positive integer sequence.
- On the third line, you must display an explicit message indicating the time taken by your algorithm, specifying the first container used to sort the positive integer sequence.
- On the last line you must display an explicit text indicating the time used by your algorithm by specifying the second container used to sort the positive integer sequence.



The format for the display of the time used to carry out your sorting is free but the precision chosen must allow to clearly see the difference between the two containers used.

Here is an **example** of standard use:

```
$> ./PmergeMe 3 5 9 7 4
Before: 3 5 9 7 4
After: 3 4 5 7 9
Time to process a range of 5 elements with std::[...] : 0.00031 us
Time to process a range of 5 elements with std::[...] : 0.00014 us
$> ./PmergeMe `shuf -i 1-100000 -n 3000 | tr "\n" " "`
Before: 141 79 526 321 [...]
After: 79 141 321 526 [...]
Time to process a range of 3000 elements with std::[...] : 62.14389 us
Time to process a range of 3000 elements with std::[...] : 69.27212 us
$> ./PmergeMe "-1" "2"
Error
$> # For OSX USER:
$> ./PmergeMe `jot -r 3000 1 100000 | tr '\n' ' '`
[...]
$>
```



The indication of the time is deliberately strange in this example. Of course you have to indicate the time used to perform all your operations, both the sorting part and the data management part.



Warning: The container(s) you used in the previous exercises are forbidden here.



The management of errors related to duplicates is left to your discretion.